



資料結構

Data Structure

Lab 09

姓名: 盧敬恩
學號: 114310123

Lab09-Ex1. (binary Tree)

搜尋,刪除,插入到二元樹

Code

```
#include <iostream>
using namespace std;
// 定義樹的節點
class TreeNode {
public:
    int data;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};
// 定義二元搜尋樹
class BinarySearchTree {
private:
    TreeNode* root;
    // 插入
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) { //如果指標指向NULL表示沒有值, 可以存放NODE
            return new TreeNode(val); //回傳
        }
        if (val < node->data) { //如果被插入的NODE>本次給的VAL
            node->left = insert(node->left, val); //大於放左下
        } else if (val > node->data) { //如果被插入的NODE<本次給的VAL
            node->right = insert(node->right, val); //小於放右下
        }
        return node; //用於遞迴
    }
    // 搜尋-----
    bool search(TreeNode* node, int target) {
        if (node == nullptr) return false;
        if (node->data == target) return true;
        if (target < node->data) return search(node->left, target);
        return search(node->right, target);
    }
    // 中序輸出-----
    void inOrderTraversal(TreeNode* node) {
        if (node == nullptr) return;
        inOrderTraversal(node->left);
        cout << node->data << " ";
        inOrderTraversal(node->right);
    }
    // 尋找最小值節點 (刪除時尋找後繼者)
    TreeNode* findMin(TreeNode* node) {
        while (node && node->left != nullptr) { //找到
            node = node->left;
        }
        return node;
    }
    // 刪除節點-----
    TreeNode* deleteNode(TreeNode* node, int val) {
        //沒找到節點
        if (node == nullptr) return nullptr;
```

```

//尋找目標節點
if (val < node->data) {
    // 要刪除的值比較小, 往左走
    node->left = deleteNode(node->left, val);
} else if (val > node->data) {
    // 要刪除的值比較大, 往右走
    node->right = deleteNode(node->right, val);
    //找到要刪除的
} else {
    if (node->left == nullptr) {
        // 如果左邊是空的, 把右子樹往上提
        TreeNode* temp = node->right;
        delete node;
        return temp; // 回傳右子樹地址給父節點接住
    } else if (node->right == nullptr) {
        // 如果右邊是空的, 把左子樹往上提
        TreeNode* temp = node->left;
        delete node;
        return temp; // 回傳左子樹地址給父節點接住
    }
    // 右子樹的最小值來當繼承者
    // 1. 呼叫 findMin 進入右子樹直到找到左葉
    TreeNode* temp = findMin(node->right);
    //繼承到當前節點
    node->data = temp->data;
    //這時樹中會有兩個重複的值, 所以要去右子樹把原本後繼節點刪除
    node->right = deleteNode(node->right, temp->data);
}
// 完成判斷回傳節點
return node;
}
//-----
public:
    BinarySearchTree() : root(nullptr) {}
    void insert(int val) {
        root = insert(root, val);
    }
    bool search(int target) {
        return search(root, target);
    }
    void displayInOrder() {
        inOrderTraversal(root);
        cout << endl;
    }
    void deleteValue(int val) {
        root = deleteNode(root, val);
    }
};

```

```

int main() {
    BinarySearchTree bst; //宣告2元搜尋樹
    int values[] = {7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5};
    int n = sizeof(values) / sizeof(values[0]); //設定N為初始TREE節點的數量
    for (int i = 0; i < n; ++i) { //加入樹中
        bst.insert(values[i]);
    }
    int key;
    cout << "請輸入要搜尋/新增的鍵值: ";
    cin >> key; //key為輸入的新增/搜尋值
    if (bst.search(key)) {
        cout << "值 " << key << " 已存在於樹中, 未新增." << endl;
    } else {
        cout << "值 " << key << " 不存在, 將加入樹中." << endl;
        bst.insert(key);
    }
    cout << "中序結果: ";
    bst.displayInOrder();
    int deleteKey;
    cout << "請輸入要刪除的數: ";
    cin >> deleteKey;
    if (bst.search(deleteKey)) {
        bst.deleteValue(deleteKey);
        cout << "刪除後的樹: ";
        bst.displayInOrder();
    }
    else {
        cout << "不在樹中";
    }
    return 0;
}

```

Result

The screenshot shows a web-based C++ compiler interface. The code editor on the left contains the implementation of a Binary Search Tree. The output window on the right shows the program's execution results, which match the expected behavior described in the text: searching for 2 returns true, inserting 7 returns false, and the in-order traversal is 1 2 4 5 7 8 9 11 12 13 15. A system status overlay in the bottom right corner displays performance metrics like FPS, GPU temperature, and power usage.

Online C++ Compiler

main.cpp

```
1 #include <iostream>
2 using namespace std;
3 // 定義樹的節點
4 class TreeNode {
5 public:
6     int data;
7     TreeNode* left;
8     TreeNode* right;
9     TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
10 };
11 // 定義二元搜尋樹
12 class BinarySearchTree {
13 private:
14     TreeNode* root;
15     // 插入方法
16     TreeNode* insert(TreeNode* node, int val) {
17         if (node == nullptr) { // 如果節點指向NULL表示沒有值，可以存放NODE
18             return new TreeNode(val); // 回傳(新的)NODE
19         }
20         if (val < node->data) { // 如果插入的NODE<本次給的VAL
21             node->left = insert(node->left, val); // 大於放在左下
22         } else if (val > node->data) { // 如果插入的NODE>本次給的VAL
23             node->right = insert(node->right, val); // 小於放在右下
24         }
25         return node; // 用於遞迴
26     }
27     // 搜尋
28     bool search(TreeNode* node, int target) {
29         if (node == nullptr) return false;
30         if (node->data == target) return true;
31         if (target < node->data) return search(node->left, target);
32         return search(node->right, target);
33     }
34     // 中序輸出
35     void inOrderTraversal(TreeNode* node) {
36         if (node == nullptr) return;
37         inOrderTraversal(node->left);
38         cout << node->data << " ";
39         inOrderTraversal(node->right);
40     }
41     // 尋找最小值節點 (刪除時尋找後繼者)
42     TreeNode* findMin(TreeNode* node) {
43         while (node->left != nullptr) { // 找到
44             node = node->left;
45         }
46         return node;
47     }
48 
```

Output

請輸入要搜尋/新增的數值: 16
值 16 不存在，將加入樹中。
中序結果: 1 2 4 5 7 8 9 11 12 13 15 16
請輸入要刪除的數: 8
刪除後的樹: 1 2 4 5 7 9 11 12 13 15 16

=== Code Execution Successful ===

FPS

無

GPU 電壓

5 W

GPU 溫度

39 °C

GPU 風扇

0 RPM

Online C++ Compiler

main.cpp

```
1 #include <iostream>
2 using namespace std;
3 // 定義樹的節點
4 class TreeNode {
5 public:
6     int data;
7     TreeNode* left;
8     TreeNode* right;
9     TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
10 };
11 // 定義二元搜尋樹
12 class BinarySearchTree {
13 private:
14     TreeNode* root;
15     // 插入方法
16     TreeNode* insert(TreeNode* node, int val) {
17         if (node == nullptr) { // 如果節點指向NULL表示沒有值，可以存放NODE
18             return new TreeNode(val); // 回傳(新的)NODE
19         }
20         if (val < node->data) { // 如果插入的NODE<本次給的VAL
21             node->left = insert(node->left, val); // 大於放在左下
22         } else if (val > node->data) { // 如果插入的NODE>本次給的VAL
23             node->right = insert(node->right, val); // 小於放在右下
24         }
25         return node; // 用於遞迴
26     }
27     // 搜尋
28     bool search(TreeNode* node, int target) {
29         if (node == nullptr) return false;
30         if (node->data == target) return true;
31         if (target < node->data) return search(node->left, target);
32         return search(node->right, target);
33     }
34     // 中序輸出
35     void inOrderTraversal(TreeNode* node) {
36         if (node == nullptr) return;
37         inOrderTraversal(node->left);
38         cout << node->data << " ";
39         inOrderTraversal(node->right);
40     }
41     // 尋找最小值節點 (刪除時尋找後繼者)
42     TreeNode* findMin(TreeNode* node) {
43         while (node->left != nullptr) { // 找到
44             node = node->left;
45         }
46         return node;
47     }
48 
```

Output

請輸入要搜尋/新增的數值: 3
值 3 不存在，將加入樹中。
中序結果: 1 2 3 4 5 7 8 9 11 12 13 15
請輸入要刪除的數: 9
刪除後的樹: 1 2 3 4 5 7 8 11 12 13 15

=== Code Execution Successful ===

FPS

無

GPU 電壓

6 W

GPU 溫度

39 °C

GPU 風扇

0 RPM

